Data Science

Data Preparation for Machine Learning

Data preparation is one of the most critical steps in any ML pipeline — often taking up **60–80% of a project's time.**

structured walkthrough of the process:

Data Preparation for Machine Learning

1. Data Collection

- Gather data from relevant sources (databases, APIs, web scraping, surveys, etc.)
- Ensure data is **representative** of the problem you're solving
- Define the target variable (label) early on

Data Preparation for Machine Learning

2. Exploratory Data Analysis (EDA)

- Understand the shape, types, and distribution of data
- Identify patterns, correlations, and anomalies
- **Key tools:** pandas, matplotlib, seaborn, describe(), value_counts()

Data Preparation for Machine Learning

3. Data Cleaning

Create new features from existing ones (e.g., extracting day/month from a date)
Combine features that carry more meaning together
Bin continuous variables into categorical ranges when appropriate
Polynomial features for capturing non-linear relationships

Data Preparation for Machine Learning

4. Feature Engineering

Handle missing values — drop rows/columns, or impute using mean, median, mode, or model-based methods
Remove duplicates — detect and eliminate redundant records
Fix inconsistencies — standardize formats (dates, units, categories)
Handle outliers — detect using IQR/Z-score, then cap, remove, or transform them
Correct errors — fix typos, wrong labels, or impossible values

Data Preparation for Machine Learning

5. Data Transformation

- **Encoding categorical variables:**
    - Label Encoding — for ordinal categories
    - One-Hot Encoding — for nominal categories
- **Scaling/Normalization:**
    - Min-Max Scaling — scales to [0, 1]
    - Standardization (Z-score) — mean=0, std=1
    - Log Transform — for skewed distributions
- **Text data** — tokenization, stemming, TF-IDF, embeddings
- **Image data** — resizing, normalization, augmentation

Data Preparation for Machine Learning

6. Feature Selection

- Remove irrelevant or redundant features to reduce noise and overfitting
- Techniques:

Filter methods — correlation, chi-square, variance threshold
Wrapper methods — RFE (Recursive Feature Elimination)
Embedded methods — Lasso, feature importance from tree models

** feature is a variable

Data Preparation for Machine Learning

7. Handling Class Imbalance

Applies to classification problems with skewed class distribution
Strategies:

Oversampling — SMOTE, random oversampling
Undersampling — reduce majority class
Class weights — penalize misclassification of minority class

Data Preparation for Machine Learning

8. Data Splitting

- Divide data into:
  - **Training set** (~70–80%) — model learns from this
  - **Validation set** (~10–15%) — tune hyperparameters
  - **Test set** (~10–15%) — final unbiased evaluation
- Use **stratified splitting** for imbalanced datasets
- Use **time-based splitting** for time-series data

Data Preparation for Machine Learning

9. Data Pipelines & Reproducibility

- Divide data into:
  - **Training set** (~70–80%) — model learns from this
  - **Validation set** (~10–15%) — tune hyperparameters
  - **Test set** (~10–15%) — final unbiased evaluation
- Use **stratified splitting** for imbalanced datasets
- Use **time-based splitting** for time-series data

Data Preparation for Machine Learning

Summary Flow

```
Raw Data → EDA → Cleaning → Feature Engineering
       → Transformation → Feature Selection
       → Imbalance Handling → Train/Val/Test Split → Model Ready ✅
```

> 💡 **Key Principle:** All preprocessing steps should be learned/fitted **only on the training set** and then applied to validation and test sets — never the other way around.

Using ML/DL in Scikit-Learn

1. Install & Import Libraries

The first step is setting up your environment by installing scikit-learn along with supporting libraries like NumPy and Pandas. These libraries work together — NumPy handles numerical operations, Pandas manages data in table form, and scikit-learn provides all the ML tools you need.

Using ML/DL in Scikit-Learn

2. Load & Prepare the Data

You bring your data into the environment, either from a file like CSV, a database, or scikit-learn's built-in sample datasets. At this stage, you separate your data into two parts — the **features (X),** which are the input variables the model will learn from, and the **target (y),** which is the output or answer you want the model to predict.

Using ML/DL in Scikit-Learn

3. Split the Data

Before doing anything else, you divide your dataset into a **training set** and a **test set**. The training set is what the model learns from, and the test set is kept completely separate to evaluate how well the model performs on unseen data. A common split is 80% for training and 20% for testing. For imbalanced datasets, stratified splitting is used to ensure both sets have proportional class representation.

Using ML/DL in Scikit-Learn

4. Preprocess the Data

Raw data is rarely ready for a model. In this step you clean and transform the data so the model can understand it properly. This includes handling missing values by filling them in with a sensible substitute like the mean or median, and scaling the numerical features so that no single variable dominates others just because it has larger numbers. Crucially, all preprocessing rules must be **learned from the training data only** and then applied to the test data — never the other way around, to avoid data leakage.

Using ML/DL in Scikit-Learn

6. Train the Model

Training is the process of the model learning patterns from your training data. You feed the algorithm both the features and the correct answers, and it adjusts its internal parameters to minimize prediction errors. This is often called "fitting" the model to the data.

Using ML/DL in Scikit-Learn

7. Make Predictions

Once trained, the model is used to predict outcomes on the test set — data it has never seen before. For classification tasks, it can also output probability scores for each class, which is useful when you need to understand the model's confidence level.

Using ML/DL in Scikit-Learn

8. Evaluate the Model

You measure how well the model performed by comparing its predictions against the actual known answers in the test set. The metrics you use depend on the problem type. For classification, common metrics are accuracy, precision, recall, F1-score, and the confusion matrix. For regression, you use metrics like Mean Squared Error (MSE), Mean Absolute Error (MAE), and R^2 score. These numbers tell you where the model is doing well and where it is failing.

Using ML/DL in Scikit-Learn

9. Build a Pipeline

A pipeline is a way to chain all your preprocessing steps and your model together into a single unified workflow. Instead of applying each step manually one by one, the pipeline handles everything in the correct order automatically. This is considered best practice because it prevents data leakage, makes your code cleaner, and makes deploying the model much easier later.

Using ML/DL in Scikit-Learn

10. Hyperparameter Tuning

Every algorithm has settings called **hyperparameters** that you control — for example, the number of trees in a Random Forest or the depth of a Decision Tree. These are not learned from the data; you set them yourself. Tuning means systematically trying different combinations of these settings to find which ones produce the best model performance. Grid Search tries every possible combination, while Randomized Search samples random combinations — which is faster when there are many options.

Using ML/DL in Scikit-Learn

11. Cross-Validation

Instead of evaluating the model just once on a single test set, cross-validation splits the training data into multiple folds and trains/tests the model several times on different portions of the data. This gives you a more reliable and unbiased estimate of how your model will perform in the real world, along with a measure of how stable or consistent its performance is.

Using ML/DL in Scikit-Learn

12. Save & Load the Model

Once you are satisfied with your model's performance, you save it to disk so it can be reused without retraining. You can then load it at any time to make predictions on new data — this is the first step toward deploying the model in a real application.

Using ML/DL in Scikit-Learn

Deep Learning in Scikit-Learn

Scikit-learn supports simple neural networks through its **MLP (Multi-Layer Perceptron)** classifier and regressor. You can define the number of hidden layers, the number of neurons per layer, and the activation function. However, for more complex deep learning tasks like image recognition, natural language processing, or custom architectures, you would step outside scikit-learn and use dedicated frameworks like **TensorFlow/Keras** or **PyTorch**, which offer far more flexibility and power.

Using ML/DL in Scikit-Learn

Overall Flow Summary

The overall journey goes from **loading and understanding your data**, through **cleaning and transforming it**, then **choosing and training a model**, followed by **evaluating and tuning it**, and finally **saving it for deployment**. Each step builds on the previous one, and skipping or rushing any step typically leads to a model that performs poorly in the real world.

An Example of Data Preparation for ML

We will use data from
Titanic - Machine Learning from Disaster

Titanic - Machine Learning from Disaster | Kaggle

An Example of Data Preparation for ML

And all of this made by Claude

can you give me an example for self study with data preparation (code and some explain)

Architected comprehensive data preparation tutorial with practical code examples >

Great choice for self-study! Let me walk you through a complete, realistic example using the classic **Titanic dataset** — one of the most popular datasets for learning data preparation.

An Example of Data Preparation for ML

made by Claude

The Dataset: Titanic Survival

The Titanic dataset contains information about passengers — their age, class, gender, fare, etc. — and whether they survived. Your goal is to **prepare this data** so a model can learn from it.

An Example of Data Preparation for ML

made by Claude

Step 1 — Load the Data

```python
import pandas as pd
import numpy as np

df = pd.read_csv('titanic.csv')
df.head()
```

What this does: Loads the CSV file into a DataFrame. `df.head()` shows the first 5 rows so you can see what you're working with.

You will see columns like: `PassengerId`, `Survived`, `Pclass`, `Name`, `Sex`, `Age`, `SibSp`, `Parch`, `Ticket`, `Fare`, `Cabin`, `Embarked`

An Example of Data Preparation for ML

made by Claude

Step 2 — Explore the Data (EDA)

```python
print(df.shape)           # (891, 12) → 891 rows, 12 columns
print(df.dtypes)          # data types of each column
print(df.isnull().sum())  # count of missing values per column
print(df.describe())      # statistics for numeric columns
```

What you will find:

- `Age` has **177 missing values**
- `Cabin` has **687 missing values** (too many to use easily)
- `Embarked` has **2 missing values**
- `Name` and `Ticket` are likely not useful for prediction

This step tells you what problems need to be fixed before modeling.

An Example of Data Preparation for ML

made by Claude

Step 3 — Drop Useless Columns

```python
df = df.drop(columns=['PassengerId', 'Name', 'Ticket', 'Cabin'])
```

Why: `PassengerId` is just a row number. `Name` and `Ticket` are unique per person — a model can't learn patterns from them. `Cabin` has too many missing values (77%) to be reliable.

An Example of Data Preparation for ML

made by Claude

Step 4 — Handle Missing Values

```python
# Fill missing Age with the median age
df['Age'] = df['Age'].fillna(df['Age'].median())

# Fill missing Embarked with the most common port
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])

print(df.isnull().sum())  # Should now show 0 missing values
```

Why median for Age? The median is more resistant to extreme outliers than the mean. If a few people were very old, the mean would be pulled upward — the median is safer.

Why mode for Embarked? It's a categorical column (S, C, Q). The safest guess is the most common value.

An Example of Data Preparation for ML

made by Claude

Step 5 — Encode Categorical Variables

```python
# Convert 'Sex' to numbers: male = 0, female = 1
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})

# One-Hot Encode 'Embarked' (S, C, Q → 3 new columns)
df = pd.get_dummies(df, columns=['Embarked'], drop_first=True)

print(df.head())
```

Why encode? Machine learning models only understand numbers. `Sex` has 2 values so we use label encoding (0/1). `Embarked` has 3 values (S, C, Q) so we use one-hot encoding — this creates new columns like `Embarked_Q` and `Embarked_S` with 0 or 1 values.

`drop_first=True` removes one of the new columns to avoid redundancy (if `Embarked_Q=0` and `Embarked_S=0`, we already know it's C).

An Example of Data Preparation for ML

made by Claude

Step 6 — Feature Engineering (Optional but Useful)

python

```python
# Create a new feature: total family size on board
df['FamilySize'] = df['SibSp'] + df['Parch'] + 1

# Create a binary feature: is the person alone?
df['IsAlone'] = (df['FamilySize'] == 1).astype(int)

# Drop the original columns since FamilySize captures them
df = df.drop(columns=['SibSp', 'Parch'])
```

Why? A person traveling with 4 family members is very different from someone traveling alone. Creating a single `FamilySize` feature captures this better than two separate columns.

An Example of Data Preparation for ML

made by Claude

Step 7 — Separate Features and Target

```python
X = df.drop(columns=['Survived'])   # all features
y = df['Survived']                   # what we want to predict

print(X.shape)   # (891, 9)
print(y.shape)   # (891,)
```

Why separate them? The model needs to know what to predict (`y`) and what to learn from (`X`). Never mix them.

An Example of Data Preparation for ML

made by Claude

Step 8 — Split into Train and Test

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(X_train.shape)  # (712, 9)
print(X_test.shape)   # (179, 9)
```

Why split first before scaling? You must split before scaling. If you scale the whole dataset first, the test set influences the scaler — that is called **data leakage**, and it makes your model results overly optimistic.

`stratify=y` ensures the 0/1 ratio of Survived is the same in both sets.

An Example of Data Preparation for ML

made by Claude

Step 9 — Scale Numeric Features

python

```python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)   # learn + apply on train
X_test  = scaler.transform(X_test)        # only apply on test (⚠️ never fit!)
```

Why scale? `Age` ranges from 0–80 while `Fare` ranges from 0–500. Without scaling, the model might think `Fare` is more important just because it has bigger numbers. StandardScaler brings everything to a mean of 0 and standard deviation of 1.

An Example of Data Preparation for ML

made by Claude

Step 10 — Verify and Train a Model

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

You should see around 80–83% accuracy, which is a solid result for a clean, well-prepared dataset. Any issues in preparation — missing value handling, encoding errors, data leakage — will immediately show up as a lower score here.

An Example of Data Preparation for ML

made by Claude

Full Pipeline Version (Clean & Production-Ready)

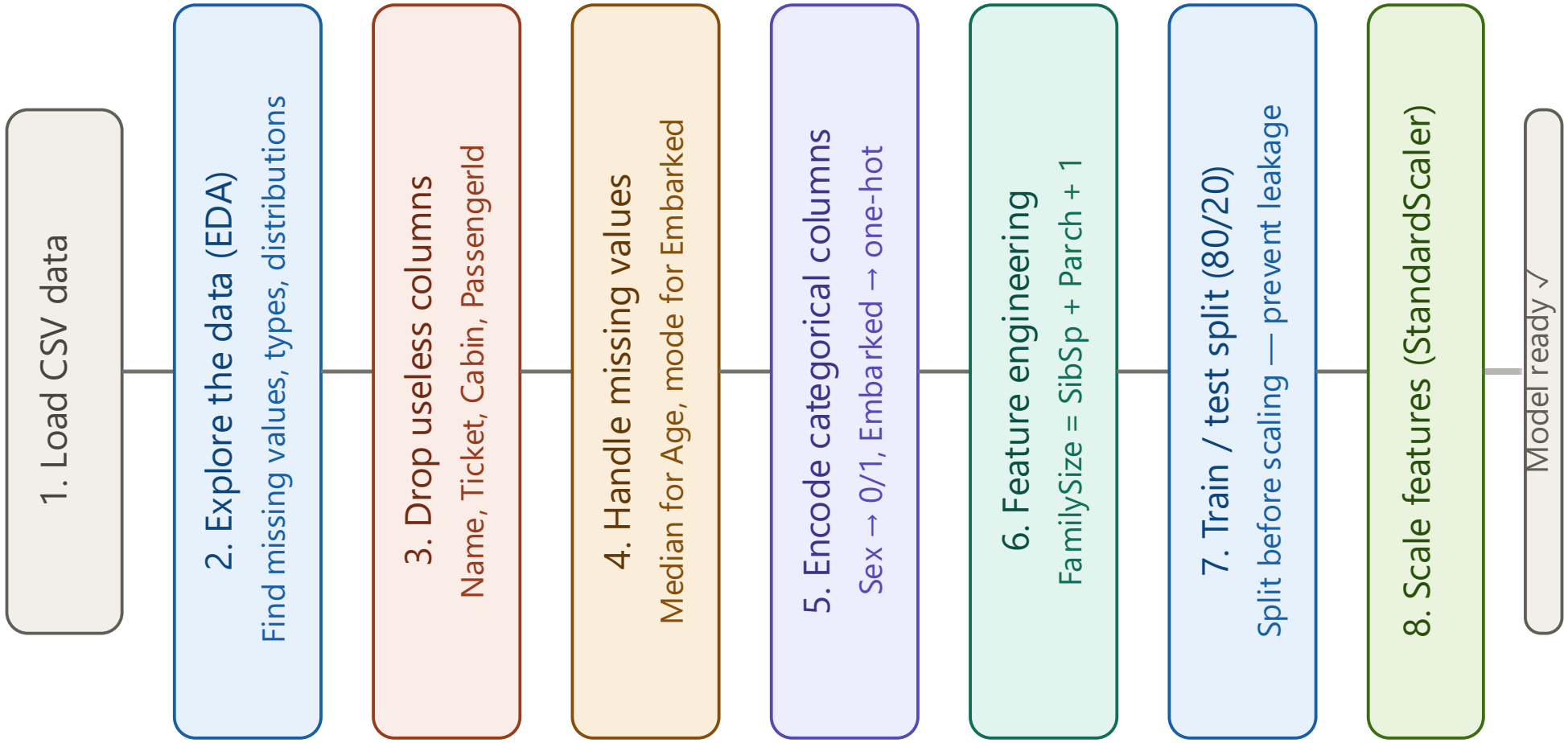
python

```python
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler',  StandardScaler()),
    ('model',   RandomForestClassifier(n_estimators=100, random_state=42))
])

pipe.fit(X_train, y_train)
print("Pipeline Accuracy:", accuracy_score(y_test, pipe.predict(X_test)))
```

This chains everything into one object — clean, reusable, and safe from data leakage.

An Example of Data Preparation for ML

made by Claude



Using Machine Learning Model

made by Claude

what about an example of using ML it doesn't cover it yet right?

Orchestrated ML modeling example using Titanic dataset >

You are right! We covered **data preparation** with the Titanic example but never showed the full **ML modeling part**. Let me continue from where we left off using the same Titanic dataset — end to end.

Using Machine Learning Model

made by Claude

Step 1 — Prepare the Data (Quick Recap)

python

```python
import pandas as pd
import numpy as np
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Load
df = sns.load_dataset('titanic')

# Drop useless columns
df = df.drop(columns=['deck', 'embark_town', 'who', 'adult_male', 'alive', 'alone'])
```

Using Machine Learning Model

made by Claude

```python
# Drop rows with missing target
df = df.dropna(subset=['survived'])

# Fill missing values
df['age'] = df['age'].fillna(df['age'].median())
df['embarked'] = df['embarked'].fillna(df['embarked'].mode()[0])

# Encode categorical columns
df['sex'] = df['sex'].map({'male': 0, 'female': 1})
df = pd.get_dummies(df, columns=['embarked'], drop_first=True)

# Features and target
X = df.drop(columns=['survived'])
y = df['survived']

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Using Machine Learning Model

made by Claude

```
# Scale
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)
```

Data is now clean and ready. From here we move into the ML part.

Using Machine Learning Model

made by Claude

Step 2 — Train Multiple Models

Instead of picking just one model, it is good practice to **try several** and compare them.

python

```python
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
```

Using Machine Learning Model

made by Claude

```
models = {
    'Logistic Regression'   : LogisticRegression(random_state=42),
    'Decision Tree'         : DecisionTreeClassifier(random_state=42),
    'Random Forest'         : RandomForestClassifier(n_estimators=100, random_state=
    'Gradient Boosting'     : GradientBoostingClassifier(random_state=42),
    'SVM'                   : SVC(random_state=42),
    'KNN'                   : KNeighborsClassifier()
}

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    print(f"{name:25s} → Accuracy: {acc:.4f}")
```

Using Machine Learning Model

made by Claude

What you will see (approximate results):

```
Logistic Regression       → Accuracy: 0.8101
Decision Tree             → Accuracy: 0.7821
Random Forest             → Accuracy: 0.8268
Gradient Boosting         → Accuracy: 0.8380
SVM                       → Accuracy: 0.8268
KNN                       → Accuracy: 0.7877
```

Random Forest and Gradient Boosting tend to perform best here. We will go with **Random Forest** for the rest of the example.

Using Machine Learning Model

made by Claude

Step 3 — Evaluate the Best Model Properly

Accuracy alone is not enough. You need a fuller picture.

python

```python
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_auc_score
)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
y_pred  = model.predict(X_test)
y_proba = model.predict_proba(X_test)[:, 1]
```

Using Machine Learning Model

made by Claude

```
# Full classification report
print(classification_report(y_test, y_pred, target_names=['Not Survived', 'Survived'

# Confusion matrix
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# ROC-AUC score
print("ROC-AUC Score:", roc_auc_score(y_test, y_proba))
```

Output will look like:

```
              precision    recall  f1-score   support
Not Survived       0.85      0.88      0.86       110
Survived           0.80      0.75      0.77        69

ROC-AUC Score: 0.8914
```

Using Machine Learning Model

made by Claude

What each metric means:

Precision — of all passengers the model predicted survived, how many actually did? High precision means fewer false alarms.

Recall — of all passengers who actually survived, how many did the model correctly catch? High recall means fewer missed survivors.

F1-Score — the balance between precision and recall. This is the most useful single number when your classes are imbalanced (more non-survivors than survivors in this case).

ROC-AUC — measures how well the model separates the two classes overall. A score of 1.0 is perfect, 0.5 is random guessing. Our 0.89 is quite good.

Confusion Matrix tells you exactly where the model is making mistakes — which survivors it missed and which non-survivors it incorrectly flagged.

Using Machine Learning Model

made by Claude

Step 4 — Cross-Validation (More Reliable Score)

Testing on just one split can be lucky or unlucky. Cross-validation tests on 5 different splits and averages the results.

```python
from sklearn.model_selection import cross_val_score, StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

scores = cross_val_score(
    RandomForestClassifier(n_estimators=100, random_state=42),
    X, y, cv=cv, scoring='accuracy'
)

print(f"CV Scores   : {scores}")
print(f"Mean Accuracy: {scores.mean():.4f}")
print(f"Std Dev      : {scores.std():.4f}")
```

Using Machine Learning Model

made by Claude

Output:

```
CV Scores    : [0.821  0.837  0.809  0.843  0.826]
Mean Accuracy: 0.8272
Std Dev      : 0.0122
```

The low standard deviation (0.012) means the model is **consistent and stable** — it is not just getting lucky on one particular split.

Using Machine Learning Model

made by Claude

Step 5 — Hyperparameter Tuning

Now we squeeze out more performance by finding the best settings for the model.

```python
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators' : [50, 100, 200],
    'max_depth'    : [None, 5, 10, 15],
    'min_samples_split': [2, 5, 10]
}
```

Using Machine Learning Model

made by Claude

```
grid_search = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train, y_train)

print("Best Parameters:", grid_search.best_params_)
print("Best CV Score  :", grid_search.best_score_)
```

Using Machine Learning Model

made by Claude

```
# Evaluate the best model on the test set
best_model = grid_search.best_estimator_
y_pred_best = best_model.predict(X_test)
print("Test Accuracy  :", accuracy_score(y_test, y_pred_best))
```

Output (example):

```
Best Parameters: {'max_depth': 10, 'min_samples_split': 5, 'n_estimators': 200}
Best CV Score  : 0.8398
Test Accuracy  : 0.8492
```

By tuning the hyperparameters we improved from **82.6% → 84.9%** — a meaningful gain from just adjusting settings.

Using Machine Learning Model

made by Claude

Output:

```
Feature       Importance
sex           0.2814
fare          0.2341
age           0.2103
pclass        0.1502
embarked_S    0.0412
...
```

This tells you that **sex, fare, and age** were the most important features for predicting survival — which matches historical knowledge (women and children were prioritized on lifeboats).

Using Machine Learning Model

made by Claude

Step 8 — Make a Prediction on New Data

python

```python
# Load back the model
loaded_model  = joblib.load('titanic_model.pkl')
loaded_scaler = joblib.load('titanic_scaler.pkl')

# A new passenger: 3rd class female, age 22, fare 7.25
new_passenger = pd.DataFrame([{
    'pclass'      : 3,
    'sex'         : 1,      # female
    'age'         : 22,
    'sibsp'       : 1,
    'parch'       : 0,
    'fare'        : 7.25,
    'embarked_Q'  : 0,
    'embarked_S'  : 1
}])
```

Using Machine Learning Model

made by Claude

```
new_passenger_scaled = loaded_scaler.transform(new_passenger)
prediction  = loaded_model.predict(new_passenger_scaled)
probability = loaded_model.predict_proba(new_passenger_scaled)

print("Survived?" , "Yes" if prediction[0] == 1 else "No")
print("Probability of survival:", f"{probability[0][1]:.2%}")
```

Output:

```
Survived?  Yes
Probability of survival: 63.00%
```

Using Machine Learning Model

made by Claude

Summary of the Full ML Process

Step	What You Did
Data Prep	Cleaned, encoded, scaled
Model Comparison	Tried 6 models, picked best
Evaluation	Accuracy, F1, ROC-AUC, Confusion Matrix
Cross-Validation	Confirmed model is stable
Hyperparameter Tuning	Improved accuracy further
Feature Importance	Understood what the model learned
Save & Predict	Deployed for real use

Using Machine Learning Model

made by Claude

This is the **complete data science workflow** from raw data to a working prediction. Would you like to go deeper on any specific step, or try a regression example next?

Using Machine Learning Model

made by Claude

from the save model can you save without joblib

10:03 AM

Identified alternative model-saving methods beyond joblib >

Yes! There are several ways to save a model without joblib. Here are all the main alternatives:

Using Machine Learning Model

made by Claude

1. Pickle (Python Built-in)

The most common alternative — pickle is built into Python so no extra install needed.

```python
import pickle

# Save
with open('titanic_model.pkl', 'wb') as f:
    pickle.dump(best_model, f)

# Load
with open('titanic_model.pkl', 'rb') as f:
    loaded_model = pickle.load(f)
```

`'wb'` means write binary and `'rb'` means read binary. Pickle converts the model object into a binary file. This is essentially what joblib does under the hood — joblib is just an optimized version of pickle for large numpy arrays.

Using Machine Learning Model

made by Claude

2. JSON (for Simple Models Only)

This only works for models that have simple numeric parameters, like Logistic Regression or Decision Tree — not Random Forest.

```python
import json

# Save Logistic Regression coefficients manually
model_params = {
    'coef'     : model.coef_.tolist(),
    'intercept': model.intercept_.tolist(),
    'classes'  : model.classes_.tolist()
}

with open('titanic_model.json', 'w') as f:
    json.dump(model_params, f)

# Load
with open('titanic_model.json', 'r') as f:
    loaded_params = json.load(f)
```

Why limited? Complex models like Random Forest have hundreds of trees with thousands of nodes — JSON is not designed to store that kind of structure efficiently. Best for simple models only.

Using Machine Learning Model

made by Claude

3. ONNX (Open Neural Network Exchange)

ONNX is a universal format that lets you save a scikit-learn model and use it in other languages and platforms like C++, Java, or JavaScript — not just Python.

```python
# pip install skl2onnx onnxruntime

from skl2onnx import convert_sklearn
from skl2onnx.common.data_types import FloatTensorType
import onnxruntime as rt
import numpy as np

# Save
initial_type = [('float_input', FloatTensorType([None, X_train.shape[1]]))]
onnx_model = convert_sklearn(best_model, initial_types=initial_type)

with open('titanic_model.onnx', 'wb') as f:
    f.write(onnx_model.SerializeToString())
```

Using Machine Learning Model

made by Claude

```
# Load and predict
session = rt.InferenceSession('titanic_model.onnx')
input_name  = session.get_inputs()[0].name
output_name = session.get_outputs()[0].name

prediction = session.run(
    [output_name],
    {input_name: new_passenger_scaled.astype(np.float32)}
)
print("Survived?", "Yes" if prediction[0][0] == 1 else "No")
```

Best for when you need to deploy your model outside of Python entirely.

Using Machine Learning Model

made by Claude

4. Cloudpickle (Better for Complex Objects)

Cloudpickle handles edge cases that regular pickle struggles with — like models inside pipelines or custom functions.

```python
# pip install cloudpickle

import cloudpickle

# Save
with open('titanic_model.pkl', 'wb') as f:
    cloudpickle.dump(best_model, f)

# Load
with open('titanic_model.pkl', 'rb') as f:
    loaded_model = cloudpickle.load(f)
```

Best for models wrapped in custom pipelines or when you have lambda functions or custom transformers inside your pipeline.

Using Machine Learning Model

made by Claude

Quick Comparison

Method	Easy to Use	Works for All Models	Cross-Language	Best For
Pickle	✅	✅	❌ Python only	General use, quick saving
JSON	✅	❌ Simple only	✅	Sharing params between systems
ONNX	⚠️ Medium	✅	✅	Production, cross-platform
Cloudpickle	✅	✅	❌ Python only	Complex pipelines

Using Machine Learning Model

made by Claude

Which One Should You Use?

For **self-study and learning** — use **Pickle**. It is simple, built-in, and works exactly like joblib.

For **production or deployment** — use **ONNX** if you need cross-platform support, or **Cloudpickle** if you have complex pipelines.

The main reason most people use **joblib over pickle** is simply that joblib is faster when the model contains large numpy arrays — for small models the difference is barely noticeable.

Machine Learning Terminology — Brief Explanations

Core Problem Types

Classification is when the model predicts a **category or label**. The answer is always one of a fixed set of options. For example — will this passenger survive? Yes or No. Is this email spam or not spam? What digit is in this image — 0 through 9?

Regression is when the model predicts a **continuous number**. For example — what will the house price be? How many units will we sell next month? What will the temperature be tomorrow? The answer can be any number, not just a fixed category.

Clustering is when the model **groups similar data points together** without being told what the groups are. There are no labels — the model finds the natural structure in the data itself. For example — grouping customers by shopping behavior.

Core Problem Types

Binary Classification is a classification problem with exactly **two possible outcomes** — survived or not, spam or not, fraud or not.

Multi-class Classification is a classification problem with **more than two possible outcomes** — for example predicting which of 10 handwritten digits an image shows.

Data Terminology

Feature is a single input variable that the model learns from. In the Titanic example, age, sex, and fare are all features. Also called a column, variable, or attribute.

Target is the output variable you are trying to predict. Also called the label or dependent variable. In Titanic it is the survived column.

Dataset is the full collection of data — all rows and columns together.

Sample / **Instance** / **Row** is a single data point or observation. One passenger in the Titanic dataset is one sample.

Data Terminology

Training Set is the portion of data the model learns from.

Test Set is the portion of data kept separate and only used to evaluate the model after training — the model never sees this during learning.

Validation Set is a third portion used during training to tune the model settings without touching the test set.

Data Leakage is when information from the test set accidentally influences the training process — this makes results look better than they really are and is a serious mistake to avoid.

Model Terminology

Algorithm is the mathematical method or recipe the model uses to learn patterns — for example Random Forest or Logistic Regression are algorithms.

Model is the result after an algorithm has been trained on data. It is the thing you save and use to make predictions.

Training / Fitting is the process of the model learning patterns from the training data. When you call `.fit()` in scikit-learn you are training the model.

Algorithm Names

Logistic Regression despite the name it is a classification algorithm, not regression. It predicts the probability of a binary outcome like survived or not.

Linear Regression predicts a continuous number by fitting a straight line through the data.

Decision Tree makes predictions by learning a series of yes/no questions about the features — like a flowchart of decisions.

Random Forest builds many decision trees and combines their answers — the majority vote wins. More powerful and stable than a single tree.

Algorithm Names

Gradient Boosting builds trees one at a time where each new tree tries to fix the mistakes of the previous ones. Very powerful — XGBoost and LightGBM are popular versions of this.

SVM (Support Vector Machine) finds the best boundary line that separates two classes with the maximum margin between them.

KNN (K-Nearest Neighbors) makes a prediction by looking at the K closest data points and taking a majority vote from them.

Evaluation Terminology

Accuracy is the percentage of predictions the model got right overall. Simple but can be misleading when classes are imbalanced.

Precision answers — of everything the model predicted as positive, how many were actually positive? High precision means fewer false alarms.

Recall answers — of all the actual positives, how many did the model correctly find? High recall means fewer missed cases.

F1-Score is the balance between precision and recall combined into one number. Useful when you care equally about both.

Evaluation Terminology

Confusion Matrix is a table showing exactly how many predictions were correct and where the mistakes were made — true positives, false positives, true negatives, and false negatives.

ROC-AUC measures how well the model separates the two classes across all possible thresholds. A score of 1.0 is perfect and 0.5 means the model is no better than random guessing.

Hyperparameter Terminology

Parameter is a value the model learns automatically during training — like the weights in a neural network or the split points in a decision tree.

Hyperparameter is a setting you manually choose before training — like the number of trees in a Random Forest or the depth of a decision tree. These are not learned from data.

Hyperparameter Tuning is the process of systematically searching for the best hyperparameter values to improve model performance.

Hyperparameter Terminology

Grid Search tries every possible combination of hyperparameter values you specify.

Cross-Validation splits training data into multiple folds and trains and evaluates the model several times to get a more reliable performance estimate.

Preprocessing Terminology

Normalization scales values to a range between 0 and 1.

Standardization transforms values so the column has a mean of 0 and a standard deviation of 1. This is what StandardScaler does.

Encoding converts categorical text values into numbers so the model can process them.

Preprocessing Terminology

Imputation is the process of filling in missing values with a substitute like the mean, median, or mode.

Feature Engineering is creating new features from existing ones to give the model more useful information to learn from.

Feature Selection is choosing only the most relevant features and removing the ones that add noise or redundancy.

> Think of these terms as your ML vocabulary — the more comfortable you are with them, the easier it becomes to read documentation, research papers, and other people's code. Would you like me to go deeper on any of these?